

```
1  # Name: Christopher A. Lee
2  # Date: 04/28/2026
3  # Prof: Dr. Callura
4  # Course: Bioinformatics - ENBC 455
5
6  # Libraries
7  from Bio import SeqIO
8  from Bio.Seq import Seq
9
10 # Problem 1
11 with open("hw12_1.txt", "r") as f:
12     reads = [line.strip() for line in f.readlines() if line.strip()]
13
14     genome = reads[0]
15     unused_reads = reads[1:]
16
17     while unused_reads:
18         best_overlap = -1
19         best_idx = -1
20         best_append = ""
21
22         for i, r in enumerate(unused_reads):
23             ov_forward = 0
24             for j in range(min(len(genome), len(r)), 0, -1):
25                 if genome[-j:] == r[:j]:
26                     ov_forward = j
27                     break
28
29             if ov_forward > best_overlap:
30                 best_overlap = ov_forward
31                 best_idx = i
32                 best_append = r[ov_forward:]
33
34             r_rc = str(Seq(r).reverse_complement())
35             ov_reverse = 0
36             for j in range(min(len(genome), len(r_rc)), 0, -1):
37                 if genome[-j:] == r_rc[:j]:
38                     ov_reverse = j
39                     break
40
41             if ov_reverse > best_overlap:
42                 best_overlap = ov_reverse
43                 best_idx = i
44                 best_append = r_rc[ov_reverse:]
45
46         genome += best_append
47         unused_reads.pop(best_idx)
48
49     read_length = len(reads[0])
50     wrap_overlap = 0
51     end_chunk = genome[-read_length:]
52     start_chunk = genome[:read_length]
53
54     for j in range(min(len(end_chunk), len(start_chunk)), 0, -1):
55         if end_chunk[-j:] == start_chunk[:j]:
56             wrap_overlap = j
57             break
58
59     if wrap_overlap > 0:
60         genome = genome[:-wrap_overlap]
61
62     print("Assembled Cyclic Superstring:")
63     print(genome)
```

Assembled Cyclic Superstring:  
TATGTGCAAGTTGATACCGTTGTCCTTAGCGCCGAGACAGATGGTTGCCAGTTCGTTAATACTATTAGCATATTAATGTGTGAACCATCAACACTGTGGTTTTCAACCCGTGAGCGGGTCTAGATGGTGAAAAAACCGGCTTATTATCTAACGACATAAAGGGGCAG  
GTGCGGTGTTTAAAGCGCGGGCTGGCAATGGCACTATTAACCCGAGTTCTTGTAACCTGGCAGCTGTCGGCAGATGTCAAGAGCTCCCAGATATTGCCCAGGCCAAAGGGCCAAACTGCTGCAGACAGAGGAACTCTTCGGCTCATCGCCAGAAGCTACACTTTTGATGACTAAGAC  
AATTTACTCTAGACATTGCAAACGCGTGGATCGAACAGTTGGCTACAACGTCGAGTGTCAAGCGCCTTCCGCTTTCGGCTGAGAAGCGACGAAAGCATCACCAAGCTGTGCATATCGGGGGCCATTGCTGTAGGTTCTCTTCGTTATACTAGATAATACTCAAGAGACGCAATATT  
TCACACGTTGAAGCCTGATTCTATGAAACGGACCACTATGTTTGAGTGATGACGGGTCGCAATGACTGCCGAATTAGATGAACCGTGGCCACTGACTCCTGCGTCGTACCCCCAACATCCAGGTGGCTAGTCCAAAATCGTATAGGAGAAGTAGAGAGACTTGACTACAACGTTGA  
TAACCGCGATAGAGAATACTGGCGCCTGGACTTCTTCTCATGGGGAGTCATATCTAGTCATCGGATGTATTGCTTGCCACGTATCCCCAGATAAACGACTTGAAAAATGCTCCGAGACAAACAGATAAAGCGGTACGAAGGGATTTTGTCCGGATCGGGGGGAATAGTATTATTC  
GCGTCTCACGATATTGTGTAGGCTGGGGATTGGTTCCAGTATCGGCTACGAGCGGCCCTAGTTTAGTGTTTCGACAAACTTAAGAGTTCGAGTCTTTCGACCCACCGCGAACGCTGGTTCTACCCATGCGGACTAGGTTACACAATTCCCTACGTTTAACGCTTATAGTGAATGGGA  
CATTGTGGTTGGCTATCTTGTAGATCCCCCGGTGGGAAATTGGTGCTCCGCCCCAGTAGGGATAACCTATAGAGAATGCATGCCGCTGGTGGGCACCGCTGCCGAAGACGGTGTTTGTGACGTCACCTAGTATAGCGTCGGATGCAATCACAGAGCGAATATTAATTTTTTCAGACT  
TTAGTCCCAGAGCCCAGGTCGCTGAAATCATAGCCTCCTAAAGGTGTCCCCTCGGTCGAGGACTTTGCTCTGAGCTATCACCAACAGGAAAAGTTCATGACTTTGGCGTCTTCCAAAGGGTACGGCTTCTGCGGGTAGCTACTACTTTCTGGTCTCCGCTGTAGGTAGCGGGCCAGC  
AAAACGCCTTGCCCTCCCCTATGTAGTGGCACCGAAGGCAACTCCACCCGGGTAAAGAATGATAGGGGTGAGCTCTCGACTATCAAATTCGGTGTGGAGGTGATAGAAGTTCGATAATGCCTGCCTTAAGTCCACATCACGTTGTAATCCTATAACCAGCAGACTGCTCGCACAC  
AGCCTCCACCTTGGGTAGCGGCTAACCTATAGCTGGAGGGCTTCACGTACTATGTAATTCTGGATTAGCCTATCGGTACGGACATAGGGTTCGGTGTGCGGGCCCCTGGCAACTGGCCACATCAGAGTCGACTGCCCCGTACACTTGGCGGAAGCGTTGTGGTGCATTCGTCCC  
GTGGGAGGTTGCACGAAGGCCAGATGAGCGAGTAGGACTCCTTCCCTTTGTTGTTTTGATCTTTGACCTGTATGAGGCGACAATTCGTGCTTACAATCGCCCAGGTTTTAACTAGCGATTACAGGTTCCGGTTAAATCAACATGAGTCCCGGTTCTCTTGTAGAGCTAGGAATCATT  
AAATACCCCGTATAGCCCCGGTGGGAATCCTGATTACTGAATTACTCTGTGTTTTTTGTCCAA